

Dependency Paging

Tal Zohar

May 10, 2026

1 The Dependency Paging Problem

There are n pages and a cache of size k . Additionally, we are provided with a dependency DAG G . Let $G[p]$ be the set of dependencies of page p . A valid solution must ensure that if page p is in cache, then all pages in $G[p]$ are also in the cache. Let l denote the width of G - the maximum antichain length.

We analyze the problem assuming pages are requested one by one, such that every request arrives only after all of its children (dependencies) have been requested. This topological request order simplifies the algorithm description and proofs without loss of generality.

2 The Unweighted Case - Marking Algorithms and Phase Structure

To analyze both the deterministic and randomized algorithms for the unweighted problem, we utilize the classical framework of *marking algorithms*, adapted to respect dependency constraints.

A marking algorithm processes a request sequence in a series of disjoint phases. At the beginning of a phase, all pages are considered “unmarked.” As requests arrive, the requested pages (and their dependencies) are “marked.” The fundamental rule of a marking algorithm is that it **never evicts a marked page**. When the cache is full and an eviction is required, the algorithm is forced to choose an unmarked page. A phase naturally terminates when an eviction is needed but the cache is entirely full of marked pages. At this point, all marks are cleared, and a new phase begins.

Consequently, the sequence is divided into phases containing exactly k distinct pages, where k represents the size of the cache. Let m denote the number of new pages requested in a phase (pages that were not in the cache at the start of the phase).

Lemma 1 (OPT Lower Bound). *The optimal offline algorithm (OPT) incurs an amortized cost of at least $m/2$ misses per phase.*

Proof. Because OPT has a cache capacity of exactly k , and each phase requests exactly k distinct pages including m entirely new pages, OPT must incur at least m misses across the current and previous phases combined. Thus, its amortized cost per phase is at least $m/2$. $\square$

By Lemma 1, any algorithm that incurs $O(m \cdot f(l))$ misses per phase is $O(f(l))$ -competitive.

We partition the DAG G into l disjoint strongly-ordered chains $C_1, \dots, C_l$, which is possible by Dilworth’s theorem.

Definition: A page is considered **marked** if it has been requested during the current phase.

Definition: A chain is considered **marked** if there is no unmarked page of the chain currently residing in the cache (i.e., all chain pages in the cache are marked).

Definition: A page in the cache is an **eviction candidate** if it is a maximal unmarked cached page (meaning it is unmarked, and no other page currently in the cache is dependent on it). Because requests follow a topological order, these are the only pages that can be safely evicted without violating dependency constraints.

Observation 1. Equivalence of Phase Boundaries: *By definition, a phase ends when all k pages in the cache are marked. Because the cache capacity is exactly k , this is the exact moment when no unmarked pages remain in the cache, and consequently, all l chains become marked. Therefore, a phase runs exactly as long as there are valid eviction candidates in the cache.*

Observation 2. *Within each chain, the marked pages reside at the bottom, while the evicted pages reside at the top.*

Observation 3. *Once a chain becomes marked, it never becomes unmarked during the phase, since pages fetched mid-phase are immediately marked upon entry.*

3 A Deterministic $O(\min(k, l))$ -Competitive Algorithm for the Unweighted Case

Standard LRU preserves dependency constraints and is $O(k)$ -competitive, as shown in [1]. By a tighter analysis, we can show that a generic dependency-aware marking algorithm is $O(\min(k, l))$ -competitive.

Claim 1. *Standard LRU is a valid implementation of the generic deterministic dependency-aware marking algorithm.*

Algorithm 1 describes this generic deterministic strategy. Whenever the cache is full, it simply evicts *any* valid eviction candidate.

Algorithm 1 Deterministic Marking Algorithm for Dependency Paging

```

1: Initialize: Cache  $S \leftarrow \emptyset$ , Marked set  $M \leftarrow \emptyset$ 
2: for each requested page  $p$  in the sequence do
3:   if  $|M \cup \{p\}| > k$  then                                 $\triangleright$  Phase ends when we attempt to mark beyond capacity
4:     End current phase, start new phase
5:      $M \leftarrow \emptyset$                                         $\triangleright$  Unmark all pages
6:     Mark all pages in  $S$  that are needed for  $p$ 
7:   end if
8:    $M \leftarrow M \cup \{p\} \cup G[p]$                              $\triangleright$  Mark the requested page and its dependencies
9:   while  $|S \cup \{p\}| > k$  do                                 $\triangleright$  Cache is full, eviction required
10:    Find an eviction candidate  $q \in S$ 
11:     $S \leftarrow S \setminus \{q\}$                                  $\triangleright$  Evict page  $q$ 
12:  end while
13:  if  $p \notin S$  then
14:     $S \leftarrow S \cup \{p\}$                                      $\triangleright$  Fetch the requested page into the cache
15:  end if
16: end for

```

To prove the competitiveness, we bound the number of misses on “old pages” (pages that were in the cache at the start of the phase).

Lemma 2 (Bound on Displaced Pages). *At any given moment during a phase, the total number of old pages that are currently evicted from the cache is bounded by m .*

Proof. Because the cache maintains a fixed capacity of k , every currently evicted old page must have been perfectly replaced by a “new page” requested during this phase. Since at most m new pages are requested in the entire phase, the cache can physically hold at most m new pages at any given time. Thus, the total number of evicted old pages is bounded by m . $\square$

Lemma 3 (Chain Marking Upon Miss). *Let p be an old page belonging to chain C_j that was evicted during the current phase. At the exact moment p is subsequently requested and brought back into the cache, C_j becomes a marked chain.*

Proof. Because C_j is a totally ordered chain, any other page $y \in C_j$ must either be a dependency of p , or y must depend on p .

- If y is a dependency of p , the algorithm guarantees that when p is marked, all of its dependencies currently in the cache are also marked. Thus, y is marked.
- If y depends on p , then y requires p to be in the cache. When p was originally evicted, it was an eviction candidate, meaning no cached page depended on it. Since p has remained evicted until now, no page depending on p (including y) could have been requested or fetched into the cache, as doing so would require fetching p first. Thus, y is currently not in the cache.

Therefore, every page of C_j currently in the cache is marked, making C_j a marked chain. $\square$

Theorem 1. *The generic deterministic marking algorithm is $O(\min(k, l))$ -competitive.*

Proof. We are interested in bounding the total number of misses on old pages. An old page incurs a miss if and only if it is evicted and subsequently refetched.

First, observe that once an old page is refetched, it becomes marked. By the fundamental rule of marking algorithms, marked pages are never evicted. Thus, each of the at most k old pages originally in the cache can incur a maximum of one miss per phase. This bounds the total number of misses on old pages strictly by k .

Second, we can bound the misses using the chain structure. By Lemma 3, the very first time an evicted old page from C_j is refetched, C_j becomes a marked chain. By Observation 3, a marked chain never becomes unmarked again for the remainder of the phase.

Because the algorithm only evicts unmarked pages (eviction candidates), no page from C_j can be evicted once C_j becomes marked. Therefore, all evictions from C_j must occur *before* C_j suffers its first miss.

By Lemma 2, the maximum number of pages from C_j that can be simultaneously evicted at any given moment is bounded by the total number of evicted old pages in the entire system, which is m . Since C_j cannot be evicted from after its first miss, the maximum number of misses C_j can incur is exactly the number of its pages that were evicted prior to this moment, which is at most m .

Summing this bound over all l chains, the total number of misses on old pages is also bounded by $m \cdot l$.

Combining these two independent bounds, the total number of misses on old pages is at most $\min(k, m \cdot l) \leq m \cdot \min(k, l)$ (since $m \geq 1$).

The algorithm also incurs exactly m misses to fetch the m new pages. Thus, the total cost for the phase is at most $m + m \cdot \min(k, l) = m(1 + \min(k, l))$. By Lemma 1, OPT incurs an amortized cost of at least $m/2$ per phase. Therefore, the generic deterministic marking algorithm achieves an overall competitive ratio of $O(\min(k, l))$. $\square$

4 A Randomized $O(\min(\log k, \log l))$ -Competitive Algorithm for the Unweighted Case

The paper [1] provides an expected $O(\min(\log k, \log l))$ -competitive randomized algorithm. Here, we provide an alternative, simpler analysis to establish this exact $O(\min(\log k, \log l))$ competitive ratio directly.

We present a randomized algorithm (Algorithm 2) that restricts how the eviction candidates are chosen. By ensuring evictions are distributed symmetrically across the chains, we prevent the adversary from stacking m misses on a single chain.

4.1 Blocked Chains and Algorithm Description

We reuse the chains $C_1, \dots, C_l$ and the definition of an eviction candidate. To formalize the randomized selection, we define how a chain specifies its desired eviction target.

Definition: For any unmarked chain, its **next eviction candidate** is defined as its highest unmarked page currently residing in the cache. Note that a chain's next eviction candidate might not be a global *eviction candidate*, as there could be cached pages *outside* the chain that depend on it.

We introduce the concept of a “blocked chain” to systematically determine how actual eviction candidates are chosen.

Definition: An unmarked chain is considered **blocked** on a page p if the following two conditions are met:

1. p is an eviction candidate.
2. p is an ancestor of (or equal to) the chain's next eviction candidate.

Claim 2. *Every unmarked chain must be blocked by some page (can be multiple).*

Proof. Let $\tilde{C}$ be an unmarked chain. By definition, it has a next eviction candidate $x \in \tilde{C}$ currently residing in the cache. If we trace the dependency graph upwards from x to its ancestors currently in the

cache, we will eventually reach a maximal unmarked cached page q (one with no cached dependents). This page q is an eviction candidate. Because q is either equal to x or an ancestor of x , the chain $\tilde{C}$ is blocked on q . $\square$

Eviction Rule: When we need to evict a page, we draw an unmarked chain $\tilde{C}$ uniformly at random. We select some page q such that $\tilde{C}$ is blocked on q (by Claim 2, we are guaranteed such a page exists), and we evict q .

Algorithm 2 Randomized Marking Algorithm for Dependency Paging

```

1: Initialize: Cache  $S \leftarrow \emptyset$ , Marked set  $M \leftarrow \emptyset$ 
2: Partition  $G$  into  $l$  disjoint chains  $C_1, \dots, C_l$  ▷ via Dilworth's Theorem
3: for each requested page  $p$  in the sequence do
4:   if  $|M \cup \{p\}| > k$  then ▷ Phase ends when we attempt to mark beyond capacity
5:     End current phase, start new phase
6:      $M \leftarrow \emptyset$  ▷ Unmark all pages
7:     Mark all pages in  $S$  that are needed for  $p$ 
8:   end if
9:    $M \leftarrow M \cup \{p\} \cup G[p]$  ▷ Mark the requested page and its dependencies
10:  while  $|S \cup \{p\}| > k$  do ▷ Cache is full, eviction required
11:    Draw an unmarked chain  $\tilde{C}$  uniformly at random
12:    Select a page  $q \in S$  such that  $\tilde{C}$  is blocked on  $q$ 
13:    ▷ ( $q$  is an eviction candidate, and ancestor of  $\tilde{C}$ 's next eviction candidate)
14:     $S \leftarrow S \setminus \{q\}$  ▷ Evict page  $q$ 
15:  end while
16:  if  $p \notin S$  then
17:     $S \leftarrow S \cup \{p\}$  ▷ Fetch the requested page into the cache
18:  end if
19: end for

```

4.2 Conceptual Accounting Scheme and Analysis

To bound the expected number of cache misses, we introduce a conceptual accounting scheme using “coins”. It is important to note that these coins are strictly an analytical tool used for the proof; the algorithm does not track or utilize them during its execution.

Coin Distribution Rules:

- **Giving Coins:** Whenever the algorithm evicts a page q by randomly selecting an unmarked chain $\tilde{C}$, we conceptually give one coin to chain $\tilde{C}$. We define the evicted page q to be the page “in charge” of this coin.
- **Removing Coins:** If a page q that is in charge of a coin is subsequently requested (and thus marked), we remove its associated coin from its chain. This scenario occurs exclusively when an evicted old page is refetched into the cache.

Based on this accounting scheme and the structural properties established in Section 2, we establish the following claims:

Claim 3 (Bound on Total Coins). *At any given moment, the total number of coins in the system is bounded by m .*

Proof. By our accounting rules, a coin corresponds exactly to an old page that is currently evicted. By Lemma 2, the number of currently evicted old pages is bounded by m . Thus, the total number of coins is bounded by m . $\square$

Claim 4 (Coin Removal). *We only remove coins from marked chains.*

Proof. A coin is removed from a chain C' exactly when the page p in charge of it is requested. We must show that at this moment, C' is a marked chain (i.e., it has no unmarked pages in the cache).

When p was originally evicted and took charge of the coin, C' was blocked on p . By definition, this means C' had a next eviction candidate x (its highest unmarked cached page at that time), and p was an ancestor of x (or equal to x).

Fast forward to the moment p is requested and the coin is removed. Because p is requested, p is immediately marked. The algorithm guarantees that when a page is marked, all its dependencies currently in the cache are also marked. Because p is an ancestor of x , x is a dependency of p . Thus, x becomes marked.

Could there be some other page $y \in C'$ in the cache that is unmarked? Because C' is a totally ordered chain of dependencies, y must be either a dependency of x , or x must be a dependency of y .

- If y is a dependency of x , then by transitivity, y is a dependency of p . For the same reason x is marked, y must also be marked.
- If x is a dependency of y , then y is an ancestor of x . However, recall that when p was evicted, it was an eviction candidate, meaning it had *no* cached dependents (no cached pages depending on it). Because C' is a totally ordered chain, y and p must be ordered. Since both y and p are ancestors of x , if y were an ancestor of p (meaning y depends on p), it being in the cache would violate p 's status as an eviction candidate. Thus, such an unmarked y cannot exist in the cache.

Therefore, every page of C' currently in the cache is marked. By definition, C' is a marked chain. $\square$

Let u denote the total number of unmarked chains at the beginning of the phase. By definition, an unmarked chain must possess at least one unmarked page currently residing in the cache. Because the cache capacity is exactly k , there can be at most k such chains. Since there are l chains in total, we have $u \leq \min(k, l)$.

Furthermore, by Observation 3, a marked chain never becomes unmarked during a phase. Let $C_1, C_2, \dots, C_u$ denote these initially unmarked chains, indexed in the exact order they become marked during the phase. Let t_j denote the number of coins residing on chain C_j at the exact moment it becomes marked.

Claim 5 (Expected Coins on Marked Chains). *When the j -th initially unmarked chain (C_j) becomes marked, it possesses in expectation at most $\frac{m}{u-j+1}$ coins. That is:*

$$E[t_j] \leq \frac{m}{u-j+1}$$

Proof. Consider the exact moment just before C_j becomes fully marked. At this moment, exactly $j-1$ initially unmarked chains have already been marked, meaning there are exactly $u-j+1$ unmarked chains remaining in the system (including C_j).

By Claim 4, we only remove coins from marked chains. This means there may be coins residing on chains that are already marked, but this is only in our favor, as it leaves fewer coins distributed among the remaining unmarked chains.

By Claim 3, the total number of coins in the entire system at any given moment is bounded by m . Since the algorithm distributes new coins uniformly at random among all currently unmarked chains, by symmetry, the expected number of coins residing on any specific unmarked chain is bounded by the maximum total coins in the system divided by the number of unmarked chains, $u-j+1$. Therefore:

$$E[t_j] \leq \frac{m}{u-j+1}$$

$\square$

Theorem 2. *The randomized dependency-aware marking algorithm is $O(\min(\log k, \log l))$ -competitive.*

Proof. We are interested in bounding the number of cache misses on old pages. An old page incurs a miss if and only if it was evicted and subsequently refetched into the cache.

By our accounting rules, an old page takes charge of a coin exactly when it is evicted, and this coin is discarded exactly when the page is refetched. Therefore, the total number of misses on old pages provides an upper bound that is exactly equal to the total number of coins discarded during the phase.

By Claim 4, coins are exclusively discarded from a chain once it becomes marked. Because marked chains do not receive new coins (as evictions only target unmarked chains), the maximum number of coins that can be discarded from chain C_j is bounded by t_j , the number of coins it possessed at the exact

moment it became marked. Summing these expectations over all u initially unmarked chains provides an upper bound on the total expected cost for old pages:

$$\begin{aligned}
E[\text{misses on old pages}] &\leq \sum_{j=1}^u E[t_j] \\
&\leq \sum_{j=1}^u \frac{m}{u-j+1} \quad (\text{by Claim 5}) \\
&= m \sum_{i=1}^u \frac{1}{i} \\
&= O(m \log u) \\
&= O(m \min(\log k, \log l)) \quad (\text{since } u \leq \min(k, l))
\end{aligned}$$

This bounds the expected cost of fetching old pages. Furthermore, the algorithm incurs exactly m misses to fetch the m new pages requested during the phase. Thus, the total expected cost of the algorithm for the phase is $m + O(m \min(\log k, \log l)) = O(m \min(\log k, \log l))$.

By Lemma 1, OPT incurs an amortized cost of at least $m/2$ misses per phase. Therefore, our randomized algorithm yields an overall expected competitive ratio of $O(\min(\log k, \log l))$. $\square$

5 A Deterministic $O(\min(k, l))$ -Competitive Algorithm for the Weighted Case

We now extend our results to the weighted version of the problem, where each page p has an associated fetching/eviction cost $c(p)$. To achieve an $O(\min(k, l))$ -competitive ratio in the weighted setting, we reduce our problem to an instance of Non-Linear Paging.

5.1 Reduction to Non-Linear Paging

The Non-Linear Paging problem generalizes classic paging by allowing the "size" of a set of pages in the cache to be determined by a custom, monotone function f . A cache state S is considered feasible as long as $f(S) \leq k$.

To reduce our Weighted Dependency Paging problem to Non-Linear Paging, we keep the exact same pages, request sequence, cache capacity k , and eviction costs. The only thing we need to map is the feasibility function f .

For any set of pages S , let its **closure** be the set S combined with all of its required dependencies in the DAG G . We define our feasibility function as the total number of unique pages in that closure:

$$f(S) = |\text{closure}(S)|$$

Because adding more pages to S can only increase or maintain the size of its closure, f is a valid, monotone function (note it is submodular).

Suppose the Non-Linear Paging algorithm outputs cache state S_t to serve the current request. Using our incremental page request assumption, we can assume without loss of generality that S_t satisfies all dependency constraints, since it has no reason to evict a non-maximal page (this action won't change the capacity).

5.2 Analysis of the Width Parameter

The competitive ratio of a non-linear paging algorithm is governed by its width parameter, l_{NL} , defined as the maximum cardinality of a minimally infeasible set minus one.

Lemma 4 (Size of Minimally Infeasible Sets). *Let $\mathcal{M}$ be the collection of all minimally infeasible sets for $f(S) = |\text{closure}(S)|$. For any $S \in \mathcal{M}$, the cardinality $|S|$ is bounded by $\min(k+1, l)$.*

Proof. By definition, a set $S \in \mathcal{M}$ satisfies $f(S) > k$, but any proper subset $S' \subset S$ satisfies $f(S') \leq k$.

First, we show that S must be an antichain in the dependency DAG G . Assume for contradiction that there exist two distinct pages $x, y \in S$ such that x is a dependency of y (i.e., $x \in G[y]$). Because x

is already subsumed by the closure of y , removing x from S does not alter the total closure: $\text{closure}(S \setminus \{x\}) = \text{closure}(S)$. This implies $f(S \setminus \{x\}) = f(S) > k$, which directly violates the minimality of S . Thus, no page in S can depend on another page in S , meaning S is an antichain. Since the width of G is l , it must be that $|S| \leq l$.

Second, because the cache capacity is k , and the base cardinality of any individual page's closure is at least 1, the maximum size of a minimally infeasible set cannot exceed $k + 1$.

Therefore, the maximum cardinality of $S \in \mathcal{M}$ is strictly bounded by $\min(k + 1, l)$. $\square$

Theorem 3. *There is a deterministic $O(\min(k, l))$ -competitive algorithm for the Weighted Dependency Paging problem.*

Proof. By Lemma 4, the maximum cardinality of a minimally infeasible set in our reduction is at most $\min(k + 1, l)$. Following the definition of the width parameter in non-linear paging, $l_{NL} = \max_{S \in \mathcal{M}}(|S| - 1)$. This directly implies that $l_{NL} \leq \min(k, l - 1)$.

[2] present a deterministic algorithm that achieves a tight l_{NL} -competitive ratio for general non-linear paging. By deploying their algorithm on our constructed instance, we immediately achieve an $O(l_{NL})$ -competitive ratio, which simplifies to $O(\min(k, l))$. $\square$

6 A Randomized Algorithm for the Weighted Case

In this section, we transition to the weighted version of the randomized dependency paging problem. We first examine a natural fractional relaxation and demonstrate its structural limitations.

6.1 Approach 1: Fractional Relaxation

A standard technique for designing randomized competitive algorithms is to first formulate a continuous, fractional relaxation of the problem state. In our context, we can define the state of the cache by assigning a scalar $y_i \in [0, m]$ to each chain C_i , representing how many of its pages are currently in the cache. We allow these topmost pages to be fractional as long as the total cache capacity is strictly maintained: $\sum_{i=1}^l y_i \leq k$.

Theorem 4. *The fractional relaxation cannot achieve a competitive ratio better than l .*

Proof. Consider l disjoint chains of length m with a cache capacity $k = (m - 1)l$. Let the fetching cost of the h -th page in any chain be $w_h = 2^h$.

Since $\sum_{i=1}^l y_i \leq (m - 1)l$, at any moment there exists at least one chain j such that $y_j \leq m - 1$. The adversary repeatedly requests the m -th page of such a chain $(p_{j,m})$.

To serve $p_{j,m}$, the fractional algorithm must increase y_j to m . Since the page was entirely missing ($y_j \leq m - 1$), the algorithm must fetch $p_{j,m}$ and incur a cost of at least $w_m = 2^m$. To maintain the capacity constraint, the algorithm must decrease the mass of other chains, ensuring the existence of a new chain with $y \leq m - 1$ for the next request. Over a cycle of l requests (one for each chain), the algorithm pays $l \cdot 2^m$.

In contrast, an offline algorithm (OPT) can keep $l - 1$ chains fully in the cache and leave one chain entirely empty. OPT only pays when the empty chain is requested, at which point it swaps it with another chain. For the same l requests, OPT performs at most one swap, costing:

$$\sum_{h=1}^m 2^h < 2 \cdot 2^m$$

The competitive ratio is therefore $\Omega(\frac{l \cdot 2^m}{2 \cdot 2^m}) = \Omega(l)$. $\square$

6.2 Approach 2: Direct Fractional Relaxation with Dependency Constraints

To achieve an optimal $O(\log k)$ competitive ratio independent of general non-linear paging rounding schemes, we formulate a fractional LP that explicitly captures the temporal requests and the topological constraints of the dependency DAG.

Let E be the set of directed edges in the dependency DAG, where $(p, q) \in E$ means page p depends on page q . By the rules of the cache, if dependency q is evicted, its dependent page p must also be evicted. We define our anti-cache variables as $x_{p,j}$, representing the fractional eviction of page p during

the interval $I(p, j)$. To enforce dependencies fractionally, we strictly require $x_{p,r(p,t)} \geq x_{q,r(q,t)}$ at any time t .

Let $q(S)$ be the minimum number of pages needed to be excluded from an infeasible set S such that the closure of the remaining cached pages is feasible (i.e., its total size including dependencies is at most k).

6.2.1 The LP Formulation

We define the Primal and Dual LP relaxations explicitly modeling the DAG constraints.

Primal LP:

$$\min \sum_{p \in \mathcal{P}} \sum_{j \in [n_p]} c(p) x_{p,j}$$

Subject to:

$$\begin{aligned} \sum_{p \in S \setminus \{p_t\}} x_{p,r(p,t)} &\geq q(S) \quad \forall t \in T, \forall \text{ infeasible } S \ni p_t \\ x_{p,r(p,t)} - x_{q,r(q,t)} &\geq 0 \quad \forall (p, q) \in E, \forall t \in T \\ x_{p,j} &\geq 0 \quad \forall p \in \mathcal{P}, \forall j \in [n_p] \end{aligned}$$

Dual LP:

$$\max \sum_{t \in T} \sum_{S \in \mathcal{S}(t)} y_t(S) q(S)$$

Subject to:

$$\begin{aligned} \sum_{t \in I(p,j)} \left(\sum_{S \in \mathcal{S}(t) | p \in S \setminus \{p_t\}} y_t(S) + \sum_{q: (p,q) \in E} \gamma_{p,q,t} - \sum_{r: (r,p) \in E} \gamma_{r,p,t} \right) &\leq c(p) \quad \forall p \in \mathcal{P}, \forall j \in [n_p] \\ y_t(S) &\geq 0, \gamma_{p,q,t} \geq 0 \end{aligned}$$

The dual variables $\gamma_{p,q,t}$ act as a directed "flow" along the DAG. If the algorithm is forced to fractionally evict a dependency q , it can shift the dual cost burden upwards to the dependent page p that requires it, preventing q 's dual constraint from being prematurely violated.

6.2.2 The Fractional Algorithm

We maintain a fractional state x and actively group pages into disjoint **components** K . We now explicitly manage the γ variables within these components to route dual flow and maintain strict dual feasibility while enforcing $x_p \geq x_q$.

1. **Initialization:** Start with $x_{p,j} = 0$ for all pages. Initialize all dual variables $y_t(S) = 0$ and internal flow variables $\gamma_{p,q,t} = 0$. Let the set of components $\mathcal{K}$ initially be individual pages: $\mathcal{K} = \{\{p\} : p \in \mathcal{P}\}$.
2. **On Request:** When a request p_t arrives at time t , $x_{p_t,r(p_t,t)} = 0$. Any existing component containing p_t or its dependencies is broken down back into individual pages, and their associated internal γ flows are frozen for that interval.
3. **Continuous Update:** While there is an infeasible set $S \ni p_t$ such that the primal constraint $\sum_{p \in S \setminus \{p_t\}} x_{p,r(p,t)} < q(S)$ is violated, and $x_{p,r(p,t)} < 1$ for all $p \in S \setminus \{p_t\}$:
 - Continuously increase the dual variable $y_t(S)$.
 - For each page p , its native accumulated dual weight is $Y_{p,r(p,t)} = \sum_{\tau \leq t} \sum_{S' \ni p} y_\tau(S')$.
 - For each component $K \in \mathcal{K}$, its aggregated weight is $Y_K = \sum_{p \in K} Y_{p,r(p,t)}$.
 - **Dual Flow Routing (γ update):** Within any merged component K , the dual weight must be distributed so no page violates its capacity. We continuously increase the internal flow variables $\gamma_{p,q,t}$ along the directed edges of K . Specifically, for an infinitesimal increase dY_K , we route γ such that the *effective* dual weight increment dY_v^{eff} for every page $v \in K$ satisfies:

$$dY_v^{\text{eff}} = dy_t(S) \cdot \mathbb{I}[v \in S] + \sum_{(v,u) \in E} d\gamma_{v,u,t} - \sum_{(r,v) \in E} d\gamma_{r,v,t} = \frac{c(v)}{c(K)} dY_K$$

where $c(K) = \sum_{p \in K} c(p)$. This explicitly shifts excess dual weight from cheaper dependencies to expensive dependents.

- Increase the uniform anti-cache value x_K (applied identically to all $p \in K$) exponentially using the component's total capacity:

$$x_K = \frac{1}{k} \left(\exp \left(\frac{\ln(k+1)}{c(K)} Y_K \right) - 1 \right)$$

- **Component Merging:** As dual variables grow, if x_q reaches x_p for some dependency edge $(p, q) \in E$ and x_q is growing faster, we merge them to prevent a topological violation. We replace K_p and K_q with a new component $K' = K_p \cup K_q$. Their fractional values are locked together, permanently enforcing $x_{p,r(p,t)} \geq x_{q,r(q,t)}$ and allowing γ flow between them.

Observation 4 (Continuity of Component Merging). *When two components K_p and K_q merge, their fractional value x does not jump. The merge occurs exactly when their dual densities are equal: $\frac{Y_{K_p}}{c(K_p)} = \frac{Y_{K_q}}{c(K_q)} = D$. By the mediant property of fractions, the combined density is $\frac{Y_{K_p} + Y_{K_q}}{c(K_p) + c(K_q)} = D$, strictly preserving the exponent and ensuring continuous evolution.*

6.2.3 Competitive Ratio Analysis

Theorem 5. *The fractional algorithm yields a fractional $O(\log k)$ -competitive ratio for the Weighted Dependency Paging problem.*

Proof. Step 1: Dual Feasibility and Network Flow. To utilize weak duality, we must prove the constructed dual variables y and γ form a feasible solution to the Dual LP. The left-hand side of the dual constraint for any page v is exactly its effective dual weight Y_v^{eff} .

For an individual unmerged page p , $\gamma = 0$, and the stopping condition $x_p \leq 1$ trivially ensures $\ln(k+1) \geq \frac{\ln(k+1)}{c(p)} Y_p^{\text{eff}}$, meaning $Y_p^{\text{eff}} \leq c(p)$.

For a merged component K , the algorithm explicitly routes the γ variables such that $Y_v^{\text{eff}} = \frac{c(v)}{c(K)} Y_K$. Because the algorithm guarantees $x_K \leq 1$ for the entire component, substituting this into the update rule gives:

$$1 \geq \frac{1}{k} \left(\exp \left(\frac{\ln(k+1)}{c(K)} Y_K \right) - 1 \right) \implies Y_K \leq c(K)$$

Since $Y_K \leq c(K)$, the effective dual weight for each individual page $v \in K$ is strictly bounded:

$$Y_v^{\text{eff}} = \frac{c(v)}{c(K)} Y_K \leq \frac{c(v)}{c(K)} c(K) = c(v)$$

Because dependencies q merge with dependents p only when q 's density exceeds p 's, the required flow direction inherently aligns with the valid edge orientation (p, q) (meaning q successfully offloads weight $-\gamma$ while p absorbs $+\gamma$). Thus, the explicit γ routing guarantees every dual constraint is strictly satisfied.

Step 2: Bounding the Derivative. Let P denote the fractional primal cost and D denote the dual cost. An infinitesimal increase $dy_t(S)$ increases the dual cost by $dD = q(S) dy_t(S)$. Taking the derivative of our exponential update rule with respect to Y_K gives $c(K) dx_K = \ln(k+1)(x_K + 1/k) dY_K$.

Since $dY_K = |K \cap S| dy_t(S)$, the total increase in our primal cost is:

$$dP = \sum_{K \in \mathcal{K}} c(K) dx_K = \ln(k+1) dy_t(S) \sum_{K \in \mathcal{K}} |K \cap S| \left(x_K + \frac{1}{k} \right)$$

Summing $|K \cap S|$ exactly reconstructs the individual pages in the set, allowing the grouped component costs $c(K)$ to drop out:

$$dP = \ln(k+1) dy_t(S) \sum_{p \in S \setminus \{p_t\}} \left(x_{p,r(p,t)} + \frac{1}{k} \right)$$

Step 3: Utilizing the Cache Capacity. By definition, excluding the optimal $q(S)$ pages from S results in a set S^* such that its closure is feasible ($|\text{closure}(S^*)| \leq k$). Since $S^* \subseteq \text{closure}(S^*)$, the raw remaining pages satisfy $|S| - q(S) \leq k$, or $\frac{|S|}{k} \leq \frac{q(S)}{k} + 1 \leq 2q(S)$.

During the update, the primal constraint is unmet ($\sum_{p \in S \setminus \{p_t\}} x_{p,r(p,t)} \leq q(S)$). Substituting this structural bound into the derivative:

$$dP \leq \ln(k+1)dy_t(S) \left(\sum_{p \in S \setminus \{p_t\}} x_{p,r(p,t)} + \frac{|S| - 1}{k} \right)$$

$$dP \leq \ln(k+1)dy_t(S) (q(S) + 2q(S)) = 3\ln(k+1) \cdot q(S)dy_t(S)$$

Conclusion. Substituting $dD = q(S)dy_t(S)$, we establish $dP \leq 3\ln(k+1) \cdot dD$. Integrating this over the request sequence yields a total fractional primal cost strictly bounded by the final dual cost:

$$P \leq 3\ln(k+1) \cdot D$$

Because Step 1 established explicit dual feasibility via the γ routing, weak duality perfectly applies: $D \leq \text{OPT}$. Therefore, the fractional cost is bounded by $P \leq 3\ln(k+1) \cdot \text{OPT}$, yielding an expected $O(\log k)$ fractional competitive ratio. $\square$

6.2.4 The Necessity of the Subset Constraints

One might wonder why the Primal LP requires an exponential number of constraints—one for every infeasible subset $S \in \mathcal{S}(t)$ —rather than simply enforcing a single global cache capacity constraint alongside the explicit dependency constraints ($x_p \geq x_q$).

The Global Capacity Constraint: Notice that our formulation inherently includes the global capacity constraint. If we let $S = \mathcal{P}$ (the set of all pages in the system), the number of pages we must exclude to reach the cache capacity k is $q(\mathcal{P}) = |\mathcal{P}| - k$. The subset constraint for $S = \mathcal{P}$ becomes $\sum_{p \in \mathcal{P}} x_p \geq |\mathcal{P}| - k$, which seamlessly rearranges to the standard capacity limit: $\sum_{p \in \mathcal{P}} (1 - x_p) \leq k$.

The Inadequacy of the Global Constraint: To see why the global capacity constraint alone is insufficient, consider the following counterexample. Suppose we have a cache of capacity k . There is a set B of $k-1$ "bottom" pages, and a set T of $k-1$ "top" pages. The dependencies are configured such that *every* top page $t \in T$ depends on *all* $k-1$ bottom pages in B .

In any valid integral solution, caching even a single top page t requires caching all $k-1$ bottom pages. This consumes $1 + (k-1) = k$ slots, perfectly filling the cache. Therefore, **every integer solution must include at most 1 top page**, meaning at least $k-2$ top pages must be entirely evicted ($x_t = 1$).

Now, suppose we formulate a simplified LP using *only* the global capacity constraint ($\sum_{p \in \mathcal{P}} x_p \geq (2k-2) - k = k-2$) and the dependency constraints ($x_t \geq x_b$ for all $t \in T, b \in B$).

A fractional solution could simply assign $x_p = 0.5$ to all pages in the system.

- **Global capacity is satisfied:** $\sum_{p \in \mathcal{P}} x_p = 2(k-1) \times 0.5 = k-1$. Since $k-1 \geq k-2$, the constraint holds.
- **Dependencies are satisfied:** $x_t \geq x_b \implies 0.5 \geq 0.5$.

However, this solution leaves $1 - 0.5 = 0.5$ fractional mass of *every* top page inside the cache. The total fractional mass of cached top pages is $\frac{k-1}{2}$. For $k > 3$, this vastly exceeds the logical maximum of 1, allowing the algorithm to "cheat" by hiding an impossible combination of fractional top pages inside the global capacity pool.

How Our Subset Constraints Fix the Issue: Our LP prevents this by enforcing $\sum_{p \in S} x_p \geq q(S)$ for *every* infeasible subset.

Consider the subset $S = T$ (consisting solely of the $k-1$ top pages). What is $q(T)$? To make any subset $T' \subseteq T$ feasible, its closure must be at most k . Because the closure of any top page immediately brings all $k-1$ bottom pages into the cache, we can retain at most 1 top page. To make $S = T$ feasible, we are forced to exclude $q(T) = |T| - 1 = k-2$ pages.

This generates the specific LP constraint:

$$\sum_{t \in T} x_t \geq k-2$$

If we evaluate the cheating fractional solution ($x_t = 0.5$), the sum is $\frac{k-1}{2}$. For any $k > 3$ (e.g., $k = 5$), the sum evaluates to 2, which strictly violates the required bound of $k-2 = 3$.

References

- [1] Julien Dallot et al. “Dependency-aware online caching”. In: *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE. 2024, pp. 871–880.
- [2] Ilan Doron-Arad, Joseph, and Naor. “Non-Linear Paging”. In: *arXiv* (2024). DOI: [10.48550/arxiv.2404.13334](https://doi.org/10.48550/arxiv.2404.13334).